

A Complete University-Level Project Brief

AI Engineering Portfolio Project · Computer Engineering

Difficulty: Intermediate · Duration: 3–4 Weeks · GPU Required

Table of Contents

1.	Project Overview & Motivation
2.	Problem Statement
3.	Technical Background
4.	System Architecture
5.	Dataset Requirements & Strategy
6.	Evaluation Framework (Build This First)
7.	Fine-Tuning Strategy
8.	Ablation Study Plan
9.	Results & Deliverables
10.	Tools & Technologies
11.	Timeline & Milestones
12.	How to Present This in Interviews

1. Project Overview & Motivation

Large Language Models (LLMs) like GPT-4 and Mistral are trained on vast amounts of general internet text. While they perform impressively across many tasks, they often fall short in specialized domains — legal reasoning, medical diagnosis, Arabic dialect understanding, or any field with precise terminology and narrow knowledge requirements. This project addresses that gap.

The goal is to take an open-source LLM and adapt it to a specific domain through a process called fine-tuning, then rigorously measure whether the adaptation actually worked. The emphasis on measurement is what distinguishes this project from the thousands of fine-tuning tutorials that exist online — those tutorials tell you how to train a model, but rarely teach you how to prove it improved.

■ The single most important idea in this project: build your evaluation suite BEFORE you write any training code. This eval-first approach is what separates real ML engineering from tutorial reproduction.

Why this project matters for your career

Fine-tuning domain-specific models is one of the most commercially valuable AI engineering skills today. Almost every company deploying AI internally — law firms, hospitals, banks, e-commerce platforms — cannot afford to send sensitive data to closed APIs like GPT-4. Instead, they fine-tune smaller open-source models on their own data and host them privately. Engineers who can do this end to end, with proper evaluation, are in extremely high demand.

2. Problem Statement

A general-purpose LLM performs poorly on specialized domain questions because its training data, while enormous, treats every topic with equal weight. A question about Lebanese contract law or Lebanese Arabic dialect expressions competes in the same model space as billions of English Wikipedia paragraphs, Reddit posts, and code repositories.

The research question this project answers is:

"By fine-tuning an open-source LLM on a curated domain-specific dataset, can we measurably and consistently improve its performance on domain tasks compared to the base model — and which fine-tuning variables have the greatest impact on that improvement?"

Chosen Domain (recommended options)

Domain	Why It's a Good Choice	Data Availability
Lebanese Law	Near-zero existing benchmarks. High practical value.	Arabic legal texts, LRC databases
Arabic Dialect NLP	Hugely underserved in NLP research globally.	Social media, news, transcripts
Medical Triage (Arabic)	Safety-critical, measurable, novel.	Clinical guidelines, symptom datasets
Technical Support QA	Easy to collect data, clear ground truth.	Company docs, Stack Overflow

Table 1: Recommended domain choices with rationale

3. Technical Background

3.1 What is Fine-Tuning?

Pre-training an LLM from scratch requires billions of dollars of compute and months of time. Fine-tuning takes an already-trained model and continues training it on a much smaller, domain-specific dataset — updating the model's weights to shift its probability distributions toward your target domain. It is orders of magnitude cheaper and faster than pre-training, while achieving surprisingly strong domain adaptation.

3.2 What is LoRA?

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique. Instead of updating all of a model's billions of parameters, LoRA freezes the original weights and adds small trainable matrices alongside specific layers. These matrices have far fewer parameters — typically 0.1–1% of the original model — making fine-tuning feasible on a single consumer GPU.

The key LoRA hyperparameter is the rank (r). A higher rank means more trainable parameters and potentially more expressiveness, but also more risk of overfitting and longer training time. Choosing the right rank is one of the ablation variables in this project.

3.3 What is 4-bit Quantization?

Quantization reduces the numerical precision of model weights from 32-bit or 16-bit floats to 4-bit integers. This shrinks memory requirements by roughly 4–8x, allowing a 7-billion parameter model to fit on an 8GB GPU. The quality loss is typically less than 2% on most benchmarks, making it an excellent tradeoff for fine-tuning on consumer hardware.

3.4 What are Evaluation Metrics?

This project uses multiple evaluation methods in combination, since no single metric captures answer quality completely:

Metric	What It Measures	When to Use
ROUGE-1 / ROUGE-L	Word and phrase overlap between model output and reference text	All tasks with text output
Exact Match (EM)	Whether the model output exactly equals the reference	Short factual answers
F1 Score	Token-level precision and recall balance	Extraction and QA tasks
LLM-as-Judge (1–5)	GPT-4 rates answer quality vs reference	Open-ended, nuanced answers
Consistency Score	Variance across 5 identical runs	Measuring model reliability
Robustness Score	Performance on rephrased versions of same question	Testing generalization

Table 2: Evaluation metrics used in this project

4. System Architecture

The project follows a strict linear pipeline. Every phase depends on the previous one, and skipping or reordering steps — especially running training before building the eval — undermines the entire scientific validity of the project.

Step 1	Domain Selection	Choose a specific domain and clearly define the task the model must perform.
Step 2	Eval Suite Construction	Build 100–300 evaluation Q&A; pairs with ground truth answers. This is your benchmark.
Step 3	Baseline Measurement	Run the base model (no fine-tuning) on your eval suite. Record all scores.
Step 4	Training Data Collection	Collect or generate 1,000–5,000 domain-specific training examples.
Step 5	Fine-Tuning	Apply LoRA fine-tuning using Unsloth + TRL. Track training loss with Weights & Biases.
Step 6	Post-Training Evaluation	Run the same eval suite on your fine-tuned model. Compare against baseline.
Step 7	Ablation Studies	Vary one hyperparameter at a time. Build the comparison table.
Step 8	Documentation	Write the README with your results table, methodology, and honest failure analysis.

5. Dataset Requirements & Strategy

This project uses two entirely separate datasets that must never overlap: the evaluation dataset and the training dataset. Mixing them — even accidentally — invalidates your results entirely.

■ Data contamination (eval examples appearing in training) is the most common mistake in fine-tuning projects. Build your eval set first, save it, then collect training data separately.

5.1 Evaluation Dataset

The evaluation dataset is your benchmark — it never changes once built, and no training data should be sourced from the same documents it came from.

Requirement	Specification
Size	Minimum 100 examples. Ideal: 200–300.
Format	Each example has: unique ID, input question, reference answer, category tag, difficulty tag (easy/m
Balance	Distribute evenly across difficulty levels and sub-categories within your domain
Ground truth	Answers must be verifiably correct — sourced from authoritative texts, not generated by an LLM
Language	Match your deployment language (Arabic, English, or bilingual)
Coverage	Cover edge cases, ambiguous questions, and common failure modes — not just easy questions

Table 3: Evaluation dataset requirements

Where to source evaluation data

- Official legal or regulatory texts (Lebanese Code of Obligations, medical guidelines, government publications)
- Academic papers and textbooks in your domain — extract question-answer pairs manually
- Existing domain expert knowledge — write questions yourself and have a domain expert verify answers
- Structured databases (legal case summaries, medical case reports)
- Do NOT use LLM-generated answers as ground truth — they introduce the hallucinations you are trying to fix

5.2 Training Dataset

The training dataset teaches the model domain knowledge. It should be diverse, clean, and formatted as instruction-response pairs.

Requirement	Specification
Size	Start with 1,000 examples. Scale to 3,000–5,000 for best results.
Format	Instruction-following format: each example is a (question, ideal_answer) pair
Quality over quantity	1,000 high-quality, accurate examples outperform 10,000 noisy ones

No duplicates	Deduplicate aggressively — near-duplicates cause the model to memorize rather than generalize
No overlap with eval	Zero tolerance. If a document was used for eval, no training data from it.
Answer quality	Answers should be complete, accurate, and at the length/style you want the model to produce

Table 4: Training dataset requirements

Where to source training data

- Legal/medical/technical texts from the public domain — extract paragraphs and convert to Q&A; pairs
- HuggingFace Datasets — search for domain-specific datasets in your language
- Synthetic data generation using GPT-4: provide source text, ask GPT-4 to generate question-answer pairs from it, then manually verify a sample
- Wikipedia in Arabic — for general Arabic language grounding
- Academic theses and reports in your domain (check university repositories)

6. Evaluation Framework

The evaluation framework is the backbone of this project. It is what transforms a fine-tuning exercise into a credible engineering project with scientific integrity.

6.1 The Four Evaluation Dimensions

Factual Accuracy	Does the model's answer agree with the verified reference answer? Measured using ROUGE scores and exact match. This is your primary metric.
Consistency	Run the exact same prompt five times with temperature > 0. Measure the variance in answers. A good model should give essentially the same answer each time.
Robustness	Rephrase each eval question three different ways. Does the model score similarly on all three phrasings? Low robustness means the model is pattern-matching, not understanding.
LLM-as-Judge	Use GPT-4o-mini (very cheap — under \$1 for a full eval run) to rate each answer from 1 to 5 compared to the reference. This captures nuance that ROUGE cannot — correct answers phrased differently still score well.

6.2 Baseline vs Fine-Tuned Comparison Template

This is the table that goes in your README and which you reference in every interview:

Model	ROUGE-1	ROUGE-L	LLM Judge (1–5)	Consistency	Notes
Mistral-7B base	0.XX	0.XX	X.X	X%	Zero-shot, no fine-tuning
+ LoRA r=8, 1K data	0.XX	0.XX	X.X	X%	Your first fine-tuned run
+ LoRA r=16, 1K data	0.XX	0.XX	X.X	X%	Higher rank
+ LoRA r=16, 3K data	0.XX	0.XX	X.X	X%	More training data
Qwen2.5-7B base	0.XX	0.XX	X.X	X%	Alternative base model
Qwen2.5-7B + LoRA	0.XX	0.XX	X.X	X%	Best configuration

Table 5: Results comparison template — fill in your actual numbers

7. Fine-Tuning Strategy

7.1 Model Selection

GPU VRAM	Recommended Model	Quantization	Expected Training Time
8 GB	Mistral-7B-v0.3	4-bit (QLoRA)	3–5 hours / 2K examples
12 GB	Llama-3.1-8B or Qwen2.5-7B	4-bit (QLoRA)	2–4 hours / 2K examples
16 GB	Qwen2.5-14B	4-bit (QLoRA)	4–6 hours / 2K examples
24 GB	Mistral-7B (full precision)	None (bfloat16)	1–2 hours / 2K examples

Table 6: Model selection guide based on available GPU VRAM

7.2 Key Hyperparameters Explained

Hyperparameter	What It Does	Recommended Starting Value
LoRA rank (r)	Controls how many parameters LoRA adds. Higher = more capacity.	16
LoRA alpha	Scaling factor for LoRA updates. Usually set equal to r.	16
Learning rate	How fast the model updates its weights.	2e-4
Batch size	Samples processed per step. Limited by VRAM. (with grad accum 4)	2
Epochs	How many times the model sees all training data.	3
Max seq length	Maximum token length per training example.	2048
Warmup steps	Gradual learning rate increase at the start to stabilize training.	100

Table 7: Key hyperparameters and recommended starting values

7.3 Training Data Format

Every training example must follow the instruction-following format below. The exact tokens depend on your base model — Mistral uses [INST] tags, Qwen uses a different chat template. Always use the model's native chat template via the tokenizer's `apply_chat_template()` method rather than hardcoding tokens.

Mistral format example:

```
<s>[INST] What is the statute of limitations for tort claims in Lebanon? [/INST]
Under Lebanese civil law, tort claims must be filed within three years from the date the victim
becomes aware of the damage and the identity of the person responsible. This is established
under Article 166 of the Code of Obligations and Contracts. </s>
```

8. Ablation Study Plan

An ablation study systematically changes one variable at a time while keeping all others constant, to isolate which factors are actually responsible for performance changes. This section is what will set your project apart in interviews — it shows you understand the science behind the engineering.

■ When an interviewer asks 'why did you choose LoRA rank 16?' — an ablation study lets you answer with data: 'because r=8 gave 0.38 ROUGE-1 and r=16 gave 0.44, while r=32 gave 0.45 but took 3x longer to train.'

Variables to ablate (one at a time)

Variable	Values to Test	What You Expect to Learn
LoRA rank (r)	4, 8, 16, 32	Where performance plateaus relative to training time
Training set size	500, 1K, 2K, 5K	The data efficiency curve — how much data is enough?
Learning rate	1e-4, 2e-4, 5e-4	Sensitivity to LR — models vary significantly
Training epochs	1, 3, 5	When does overfitting start? Does loss on eval go up?
Base model	Mistral-7B vs Qwen2.5-7B	Which base model adapts better to your domain?
LoRA target modules	Attention only vs all linear layers	Does targeting more layers help?

Table 8: Ablation study variables and expected insights

Important rule for ablation studies

Change exactly ONE variable per run. If you change LoRA rank AND learning rate in the same run, you cannot know which variable caused any observed change in performance. This is the fundamental principle of controlled experiments.

9. Results & Deliverables

9.1 What you will produce

GitHub Repository	Clean, well-documented codebase with clear README
Eval Dataset	Published on HuggingFace Hub — a real community contribution
Baseline Results	JSON file with per-example scores from the base model
Fine-Tuned Model	LoRA adapter weights published on HuggingFace Hub
Ablation Table	CSV + chart showing all ablation run results
Results README	Clear comparison table showing improvement from baseline
Failure Analysis	A section documenting where the model still fails and why
W&B; Dashboard	Public training dashboard with loss curves and eval metrics

9.2 The Failure Analysis section (critical)

Most student projects only show successes. A failure analysis — where you document specific examples the model still gets wrong and hypothesize why — signals intellectual honesty and engineering maturity. Include at least 5–10 examples of model failures, categorized by failure type:

- Hallucination: model confidently answers with incorrect information
- Refusal: model says it cannot answer when it should be able to
- Truncation: model cuts off the answer before completing it
- Formatting errors: model produces the right content but in wrong format
- Out-of-distribution: questions slightly outside the training distribution cause degraded performance

10. Tools & Technologies

Tool	Purpose	Why This One
Unsloth	Fast fine-tuning framework	2–3x faster than vanilla HuggingFace, same results
HuggingFace PEFT	LoRA implementation	Industry standard, well-documented
TRL (SFTTrainer)	Supervised fine-tuning loop	Clean API, handles data formatting automatically
Weights & Biases	Experiment tracking	Free tier, beautiful dashboards, used in industry
HuggingFace Hub	Model & dataset hosting	Free, public, gives your project a real URL
evaluate library	ROUGE, F1, EM metrics	Standard metrics, easy to use
OpenAI API	LLM-as-judge scoring	GPT-4o-mini costs ~\$1 for a full eval run
Pandas	Results analysis	Standard data analysis tool
Matplotlib / Seaborn	Ablation charts	For visualizing your ablation results

Table 9: Tools used in this project

11. Timeline & Milestones

Phase
Week 1

Foundation

- Pick your domain — commit to one and do not change it
- Set up the Python environment and install all dependencies
- Source your evaluation data (100–300 Q&A; pairs with verified answers)
- Build eval.jsonl and lock it — this file does not change after this week
- Run the baseline eval on the base model and record all scores
- Create your W&B; project and log baseline results

Phase
Week 2

Data & First Training Run

- Collect or generate 1,000–2,000 training examples
- Deduplicate and verify training data quality (manually check 50 random samples)
- Format training data into the correct instruction-following format
- Run first fine-tuning experiment with default hyperparameters (LoRA $r=16$, $lr=2e-4$, 3 epochs)
- Run the eval suite on the fine-tuned model
- Compare results against baseline — document what improved and what did not

Phase
Week 3

Ablation Studies

- Run ablation across LoRA ranks: $r=4$, $r=8$, $r=16$, $r=32$
- Run ablation across dataset sizes: 500, 1K, 2K
- Run ablation across learning rates: $1e-4$, $2e-4$, $5e-4$
- Compile ablation table in CSV and create visualizations
- Try second base model (Qwen2.5-7B if you started with Mistral, or vice versa)

Phase
Week 4

Polish & Publish

- Write failure analysis section — 5–10 documented failure cases with categories
- Publish eval dataset to HuggingFace Hub
- Publish best LoRA adapter weights to HuggingFace Hub
- Write comprehensive README with results table, methodology, and limitations
- Make W&B; dashboard public
- Record a 3-minute demo video showing the model answering questions before and after fine-tuning

12. How to Present This in Interviews

This section is as important as the technical work. A well-built project presented poorly loses to a mediocre project presented confidently with data.

12.1 The 30-second pitch

"I fine-tuned Mistral-7B on a custom Lebanese law dataset using LoRA, with an eval-first approach — I built 200 verified Q&A pairs as a benchmark before writing any training code. The fine-tuned model improved ROUGE-L from 0.18 to 0.48 on that benchmark. I also ran ablation studies varying LoRA rank and dataset size, and published both the dataset and model weights on HuggingFace."

12.2 Expected interview questions and how to answer

Q: Why did you build the eval before training?

A: Because without a fixed benchmark, you cannot know if training helped or hurt. Building the eval first forces you to define success before you see any results — which prevents post-hoc rationalization of whatever the model happened to produce.

Q: Why LoRA instead of full fine-tuning?

A: Full fine-tuning a 7B model requires 40–80GB of VRAM. LoRA reduces this to 8–16GB with less than 2% performance penalty on most tasks. For production use cases, LoRA adapters are also much easier to version, share, and swap than full model weights.

Q: How did you prevent data contamination?

A: I built the eval dataset first from one set of source documents, then collected training data from entirely separate sources. The eval.jsonl file was locked before any training data was collected.

Q: What did the ablation study tell you?

A: The most impactful variable was dataset size — going from 1K to 3K examples improved ROUGE-L by more than doubling the LoRA rank did. This suggests the model was data-hungry rather than capacity-limited, which informed the decision to invest in more data collection rather than a larger model.

Q: Where does the model still fail?

A: It struggles with questions that require multi-step legal reasoning — citing Article A, then applying it through Article B to reach a conclusion. It also occasionally hallucinates article numbers while getting the legal principle correct. This suggests the next step would be chain-of-thought fine-tuning.

■ The failure analysis is often the most impressive part of the conversation. Most candidates can only talk about what worked. Being able to precisely characterize where your model fails — and why — signals that you understand the problem deeply, not just that you ran a tutorial.

You now have everything you need.

Start with Week 1, Day 1: pick your domain and begin building eval.jsonl.

Everything else follows from that first file.
